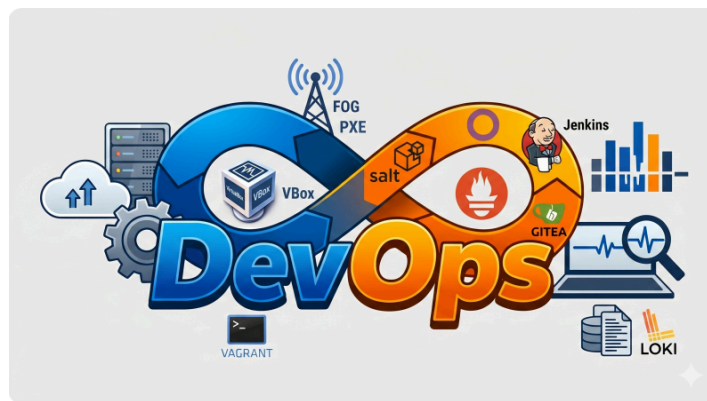


DevOps. Automatización y monitorización de infraestructuras

Unidad 06. Automatización y CI/CD: Despliegue y monitorización con Jenkins



Autor: Sergi García Barea



Actualizado Mayo 2026

Licencia



Reconocimiento – NoComercial – CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 06. Automatización y CI/CD: Despliegue y monitorización con Jenkins	3
1. 🧠 Introducción	3
1.1 Los roles en la fábrica	3
1.2 El flujo de la cadena de montaje	4
2. ⚙️ Jenkins y el hardware	4
2.1 Arquitectura interna y el almacén de planos (XML)	4
2.2 Jobs, builds y el latido del pipeline	5
2.3 Plugins: los músculos que mueven la fábrica	6
2.4 La crumb CSRF: el guardián de la API	6
2.5 DooD vs DinD: el truco del socket compartido	6
2.6 La Java Virtual Machine y el OOM Killer	7
2.7 Overlay2 y la tormenta de E/S en disco	7
3. 🏠 La aplicación: Calculadora web	7
3.1 Implementación del servicio	8
3.2 Dockerfile utilizado	8
3.3 Tests unitarios	9
4. 📋 Pipeline declarativo: la receta de montaje	9
4.1 ¿Qué es un pipeline declarativo?	9
4.2 El Jenkinsfile de nuestra fábrica	10
4.3 Las etapas del montaje (una a una)	11
4.4 ¿Cómo se entera Jenkins de que hay trabajo nuevo?	12
4.5 Limitaciones y simplificaciones de nuestro pipeline (Laboratorio)	12
5. Despliegue a producción y orquestación de la infraestructura base	12
5.1 El approval gate: la barrera humana	12
5.2 Despliegue azul/verde simulado	12
5.3 Rollback automático (una mejora opcional)	13
5.4 Terraform: el orquestador de la infraestructura de soporte	13
5.5 Separación de responsabilidades: Terraform vs Jenkins	14
6. 👁️ Observabilidad y diagnóstico de fallos	14
6.1 Prometheus: los sensores que todo lo vigilan	14
6.2 cAdvisor: el microscopio montado sobre el kernel	15
6.3 Grafana: el videowall del capataz	15
7. 📊 Métricas reales del panel de control	15

7.1 Versión desplegada en cada entorno (métrica de la aplicación)	15
7.2 Estado de la plataforma Git (Gitea)	16
7.3 Salud del pipeline Jenkins	16
7.4 Consumo de recursos (CPU y RAM) por contenedor	16
7.5 ¿Qué métricas NO están en el panel y por qué?	16
8. ★ Buenas prácticas en la implementación de pipelines CI/CD	17
8.1 Inmutabilidad absoluta	17
8.2 Pipeline como código	17
8.3 Idempotencia y determinismo	17
8.4 Separación de entornos	17
8.5 Seguridad con mínimo privilegio	17
8.6 Artefactos inmutables y etiquetado	17
8.7 Limpieza periódica de la fábrica	18
8.8 Monitorización proactiva	18
9. 📖 Webgrafía	18

UNIDAD 06. AUTOMATIZACIÓN Y CI/CD: DESPLIEGUE Y MONITORIZACIÓN CON JENKINS

1. INTRODUCCIÓN

La gestión manual de infraestructuras y despliegues presenta limitaciones de escalabilidad, repetitividad y trazabilidad. La adopción de prácticas de Integración Continua (CI) y Despliegue Continuo (CD) permite automatizar las fases de verificación, construcción y entrega de aplicaciones, reduciendo la intervención humana directa y aumentando la fiabilidad del proceso.

Un pipeline de CI/CD es una secuencia de etapas automatizadas que se ejecutan ante cada cambio en el repositorio de código. Para entender cómo funciona este pipeline en la práctica, imaginemos una fábrica de coches automatizada donde el producto final no es un vehículo, sino una calculadora web. Su simplicidad es deliberada: nos permite centrarnos en el mecanismo sin distracciones.

1.1 Los roles en la fábrica

- **El almacén de planos (Gitea):** Es el servidor Git local. Cuando un ingeniero deposita un nuevo diseño (un git push con cambios en el código), una señal eléctrica recorre la fábrica. Esa señal puede ser un webhook (aviso instantáneo) o un polling SCM (Jenkins pregunta cada dos minutos si hay novedades). En nuestro laboratorio usamos el segundo método por simplicidad, pero el primero es el estándar en entornos reales.
- **El robot capataz (Jenkins):** Es el cerebro que lee los planos —el Jenkinsfile— y decide qué máquinas activar y en qué orden. No hace el trabajo sucio directamente: su función es delegar, supervisar y detener la cadena si algo falla. Cada etapa del pipeline es una compuerta que debe abrirse antes de pasar a la siguiente.
- **Las cajas de envío (Docker):** El producto final sale sellado en contenedores inmutables. Da igual si el coche va a una pista de pruebas (entorno de staging) o a la autopista real (entorno de producción); funcionará igual porque la caja lo protege. La imagen que se construye en el pipeline es la misma que se despliega en ambos entornos.
- **El taller de pruebas (Staging):** Aquí se monta la nueva versión del coche y se somete a una inspección rápida pero severa: el smoke test. Un operario automatizado (un simple curl al endpoint /health) verifica que el motor arranca y responde. Si el test falla, la cadena se detiene en seco. No gastamos energía en promocionar basura.
- **La sala de ventas (Producción):** Es el entorno que ven los usuarios reales. Es territorio sagrado, y el acceso está protegido por un Approval Gate: una barrera que solo se abre cuando un humano —el arquitecto— revisa el panel de control y pulsa "Desplegar".
- **El panel de control (Observabilidad):** Está formado por Prometheus (los sensores que vigilan la salud de la fábrica y de las aplicaciones en tiempo real) y Grafana (un videowall que traduce métricas en gráficos visuales). Además, cAdvisor actúa como un microscopio montado sobre el demonio de Docker, leyendo desde el Kernel el uso real de RAM y CPU de cada contenedor.

1.2 El flujo de la cadena de montaje

El pipeline sigue una secuencia de etapas innegociables:

Código nuevo → Gitea → Jenkins detecta cambio

- |— 1. Checkout del repositorio
- |— 2. Tests unitarios (npm test)
- |— 3. Build de la imagen Docker
- |— 4. Deploy a Staging
- |— 5. Smoke test (health check)
- |— 6.  Aprobación manual (PD: Se puede configurar automática)
- |— 7. Deploy a Producción

Cada etapa es un eslabón. Si uno se rompe, el resto no se ejecuta. Esta cadena de responsabilidades es lo que transforma un conjunto de scripts sueltos en una auténtica “Factoría digital”.

2. JENKINS Y EL HARDWARE

Jenkins es el cerebro de nuestra fábrica. Pero un cerebro no flota en el vacío: necesita memoria RAM, ciclos de CPU y acceso al disco. Ignorar cómo sufre ese sistema durante un pipeline es la diferencia entre un arquitecto que diseña desde una torre de marfil y uno que entiende cómo optimizar las cosas y resolverlas cuando se tuercen.

2.1 Arquitectura interna y el almacén de planos (XML)

Oficialmente, Jenkins sigue un modelo Master-Agent. El **Master** es el servidor central que gestiona los pipelines, la interfaz web y la cola de trabajos. Los **Agents** son nodos remotos que ejecutan las tareas pesadas. En nuestro laboratorio, por simplicidad, usamos un único contenedor que actúa como Master y como ejecutor a la vez (la directiva agent any en el **Jenkinsfile** lo confirma). La maquinaria interna, sin embargo, es la misma.

Todo lo que Jenkins sabe está almacenado en **/var/jenkins_home/**. Ese directorio es un gran almacén de archivos XML donde cada job, cada credencial y cada plugin deja su huella:

/var/jenkins_home/

- |— **jobs/** **# Definiciones de los pipelines (config.xml)**
- |— **plugins/** **# Plugins instalados (.jpi)**
- |— **init.groovy.d/** **# Scripts de inicialización**
- |— **credentials.xml** **# Credenciales almacenadas**
- |— **config.xml** **# Configuración global**
- |— **secrets/** **# Secretos y claves**

Cuando ejecutamos docker run y montamos un volumen (jenkins-data), estamos preservando este almacén. Si el contenedor muere, la configuración sobrevive.

2.2 Jobs, builds y el latido del pipeline

Un job es una tarea configurable que Jenkins puede ejecutar. Cada ejecución se llama build y recibe un número secuencial único. Jenkins soporta tres tipos de jobs. Elegir el correcto es un acto de diseño:

Tipo	Descripción	Cuándo usarlo
Freestyle	Configuración manual vía interfaz web.	Tareas simples o scripts sueltos.
Pipeline declarativo	Definición en código (Jenkinsfile).	CI/CD complejo, la opción recomendada.
Multibranch pipeline	Un pipeline diferente por cada rama Git.	Proyectos con múltiples ramas activas.

En el laboratorio usaremos un Pipeline declarativo. El pipeline se define en un archivo llamado Jenkinsfile que viaja con el código fuente. Esto significa que el plano de montaje no está en la cabeza del capataz: está grabado en el mismo almacén de planos. Las ventajas son inmediatas: el pipeline se versiona, se revisa en el code review y se reconstruye de forma idéntica desde cualquier checkout.

La sintaxis declarativa tiene una estructura rígida y predecible:

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Test') {
      steps { sh 'npm test' }
    }
  }
  post {
    success { echo 'Pipeline completado' }
    failure { echo 'Pipeline fallido' }
  }
}
```

Cada stage es una fase. Se ejecutan secuencialmente. Si una falla, las siguientes no se ejecutan (a menos que se use post con always). No hay atajos: el código debe pasar por todas las compuertas.

2.3 Plugins: los músculos que mueven la fábrica

Jenkins se complementa mediante plugins. En nuestro laboratorio, los músculos esenciales son:

Plugin	Función
Git	Clona repositorios Git desde Gitea.
Pipeline (workflow-aggregator)	Conjunto de plugins para pipelines (CPS, SCM, API, etc.).
Docker Workflow	Construye imágenes Docker desde el pipeline.
Credentials Binding	Inyecta credenciales en variables de entorno sin exponerlas.
Prometheus	Expone métricas de Jenkins en el endpoint /prometheus.

Estos plugins no están exentos de coste. Cada uno consume memoria en la JVM, añade tiempo de arranque y puede entrar en conflicto con otros. Instalar un plugin es añadir una dependencia más al monstruo que ya respira dentro del contenedor.

2.4 La crumb CSRF: el guardián de la API

Jenkins expone una API REST para disparar builds, consultar estados o configurar jobs. Pero no acepta órdenes de cualquiera. Para protegerse de ataques Cross-Site Request Forgery (CSRF), exige un token llamado crumb. Es un talismán efímero que debe acompañar cada petición.

Si el crumb no se envía o es incorrecto, Jenkins responde con un seco HTTP 403. No hay negociación. Esta protección es necesaria, pero es una molestia cuando automatizamos desde scripts: el crumb caduca, y cada petición debe obtener uno fresco.

2.5 DooD vs DinD: el truco del socket compartido

Jenkins necesita construir imágenes Docker y ejecutar contenedores. Pero Jenkins mismo corre dentro de un contenedor. Aquí surge el dilema: ¿metemos Docker dentro de Jenkins o le prestamos el Docker del host?

- **DinD (Docker-in-Docker):** Ejecutar un demonio Docker completo dentro del contenedor de Jenkins. Suena limpio, pero es una pesadilla de rendimiento: el demonio interno consume recursos, las capas de almacenamiento se duplican y ciertas configuraciones de red se vuelven incompatibles.
- **DooD (Docker-out-of-Docker):** Montamos el archivo físico `/var/run/docker.sock` del host directamente en el contenedor de Jenkins. Este archivo es un Unix Domain Socket,

un punto de comunicación del Kernel que permite a Jenkins hablar directamente con el demonio Docker del host.

En el laboratorio usaremos DooD. El resultado es inmediato: cuando Jenkins ejecuta docker build, el comando no se ejecuta dentro del contenedor, sino que se lo pasa al servicio del host. Los contenedores creados por Jenkins son hermanos del propio Jenkins: aparecen en docker ps como procesos normales, no anidados. La eficiencia es máxima, pero el riesgo también: montar el socket equivale a entregar acceso root al host. Solo debe hacerse en entornos controlados como este laboratorio.

2.6 La Java Virtual Machine y el OOM Killer

Jenkins no es un binario ligero escrito en C o Go. Es una aplicación Java que se ejecuta sobre la Java Virtual Machine (JVM). Al arrancar, la JVM solicita al Kernel un bloque de RAM contiguo llamado Heap Memory, donde almacena objetos, cadenas y metadatos de los builds.

Cuando un pipeline pesado ejecuta npm install con miles de dependencias, Jenkins genera una cantidad masiva de logs, artefactos y referencias en memoria. Si no se han configurado límites estrictos, la JVM intenta expandir su Heap. Si el host se queda sin RAM libre (y el swap no alcanza o no existe), el Kernel de Linux invoca al implacable OOM Killer (Out-Of-Memory Killer). Este mecanismo selecciona un proceso y lo mata sin previo aviso para liberar memoria. Con frecuencia, el elegido es el propio Jenkins, que cae fulminado a mitad de un despliegue crítico.

Buenas prácticas para evitarlo: limitar la memoria del contenedor Docker (por ejemplo, --memory=2g) y configurar la JVM con -Xmx1g -Xms512m.

2.7 Overlay2 y la tormenta de E/S en disco

El comando **docker build** bajo el capó tiene un coste elevado de lectura y escritura. Docker utiliza el sistema de archivos Overlay2. Cada instrucción del Dockerfile (FROM, RUN, COPY) crea una nueva capa física en el disco. Cuando el pipeline ejecuta npm install, se escriben miles de archivos pequeños en el directorio **node_modules/**. Esta operación dispara una tormenta de operaciones de entrada/salida que satura las IOPS del disco y pone al journaling del sistema de archivos a trabajar a destajo.

El resultado es un cuello de botella invisible: el pipeline se ejecuta lentamente mientras el disco sufre en silencio. Soluciones:

- Usar un archivo **.dockerignore** para excluir node_modules, logs y otros directorios temporales.
- Minimizar el número de capas en el **Dockerfile**.
- Aprovechar la caché de capas copiando primero los archivos que cambian con menos frecuencia (como **package.json**).

3. LA APLICACIÓN: CALCULADORA WEB

La aplicación utilizada como caso de estudio es una calculadora web implementada en Node.js con Express. Su complejidad se ha reducido deliberadamente para que el foco del aprendizaje recaiga en el pipeline CI/CD y no en la lógica de negocio. La aplicación evoluciona a lo largo del

laboratorio mediante dos versiones principales: la v1 que es una aplicación para sumar y la v2 que es una aplicación para multiplicar.

El repositorio **calculator-app** contiene los siguientes archivos y directorios:

calculator-app/

```
|— src/
|   |— index.js      # Punto de entrada del servidor Express
|— tests/
|   |— app.test.js   # Pruebas unitarias con Jest
|— package.json      # Dependencias (express, jest) y scripts
|— package-lock.json # Bloqueo de versiones exactas
|— Dockerfile        # Definición de la imagen
|— Jenkinsfile       # Pipeline de CI/CD (se explica en el punto 5)
```

3.1 Implementación del servicio

El archivo `src/index.js` implementa las siguientes rutas HTTP:

- **GET /** : renderiza una interfaz HTML con dos campos numéricos (n_1 , n_2), un botón y el resultado de la operación. El método de cálculo varía según la versión.
- **GET /health** : retorna un objeto JSON con el estado del servicio. En versiones estables responde con 200 OK y `{"status":"healthy"}`. En el caso de versiones con errores (bug) responde con 500 y `{"status":"broken"}`, aunque en nuestro laboratorio, por simplicidad, no incluimos ninguna versión con errores.
- **GET /metrics** : expone una métrica Prometheus personalizada: `app_version{version="x.y.z"} 1`. Esta métrica es consumida por Prometheus y visualizada en Grafana.

3.2 Dockerfile utilizado

El Dockerfile utilizado es el siguiente:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl --fail http://localhost:3000/health || exit 1
CMD ["node", "src/index.js"]
```

Decisiones de diseño relevantes:

- La imagen base `node:20-alpine` minimiza el tamaño final (≈ 150 MB).
- La instrucción `COPY package*.json` antes que `COPY .` permite aprovechar la caché de

capas de Docker. Si package.json no cambia, npm ci no se re-ejecuta, acelerando builds sucesivos.

- npm ci --only=production instala solo dependencias de producción (omite devDependencies como Jest).
- El HEALTHCHECK integrado en la imagen permite a Docker monitorear la salud del contenedor, pero en el pipeline se utiliza un smoke test explícito con curl para mayor control.

3.3 Tests unitarios

Las pruebas unitarias se ejecutan con Jest. El archivo tests/app.test.js contiene casos muy simples que validan la operación aritmética de la versión correspondiente. Un ejemplo sencillo:

```
describe('Calculadora v2', () => {  
  test('Multiplicación básica', () => { expect(5 * 5).toBe(25); });  
});
```

4. PIPELINE DECLARATIVO: LA RECETA DE MONTAJE

El corazón de la fábrica es el Jenkinsfile. No es un script suelto que alguien escribió una tarde y dejó olvidado en un rincón del servidor. Es la receta de montaje que viaja con la propia pieza, se versiona en Git y se somete al mismo escrutinio que el resto del código fuente. Si alguien quiere cambiar cómo se ensambla la calculadora, debe modificar este archivo, hacer commit y pasar la revisión.

4.1 ¿Qué es un pipeline declarativo?

Un pipeline declarativo es un estilo de definición de flujos de trabajo en Jenkins donde la estructura está predefinida mediante bloques (**pipeline**, **stages**, **stage**, **steps**). La sintaxis es más rígida que la de un pipeline scriptado, lo que facilita la lectura y el mantenimiento. Es como un formulario que hay que rellenar: Jenkins sabe exactamente qué esperar.

Ejemplo de estructura básica mínima:

```
pipeline {  
  agent any  
  stages {  
    stage('Nombre de la etapa') {  
      steps {  
        // Aquí van los comandos  
      }  
    }  
  }  
}
```

Cada stage es una fase. Se ejecutan secuencialmente. Si falla, las siguientes no se ejecutan.

4.2 El Jenkinsfile de nuestra fábrica

El Jenkinsfile que utilizamos en el laboratorio es el siguiente (he simplificado la parte de entorno para que no aparezcan IPs concretas):

```
pipeline {
  agent any
  environment {
    APP_NAME = "calculator-app"
    // Las IPs de staging y producción se definen en el job de Jenkins
    // o se inyectan mediante variables externas
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Unit Tests') {
      agent { docker { image 'node:20-alpine'; reuseNode true } }
      steps { sh 'npm install && npm test' }
    }
    stage('Build Image') {
      steps { sh "docker build -t ${APP_NAME}:latest ." }
    }
    stage('Deploy Staging') {
      steps {
        sh "docker stop ud06-staging || true && docker rm ud06-staging || true"
        sh "docker run -d --name ud06-staging --hostname staging --network LabCEFire -e NODE_ENV=staging ${APP_NAME}:latest"
      }
    }
    stage('Smoke Test Staging') {
      steps {
        sleep time: 10, unit: 'SECONDS'
        sh "curl -s http://ud06-staging:3000/health | grep 'healthy'"
      }
    }
    stage('Aprobar despliegue a producción') {
      steps {
        input message: '¿Staging funciona correctamente?', ok: 'Desplegar'
      }
    }
    stage('Deploy Production') {
```

```

steps {
  sh "docker stop ud06-production || true && docker rm ud06-production || true"
  sh "docker run -d --name ud06-production --hostname production --network LabCEFIRE -e
NODE_ENV=production ${APP_NAME}:latest"
}
}
}
}

```

Observaciones:

- Hemos eliminado las IPs explícitas y usado nombres de host (staging, production) que resuelve la red Docker interna.
- La etiqueta latest es una simplificación. En un entorno real se usaría el número de build o el hash del commit.
- El input detiene el pipeline hasta que un humano pulse "Desplegar". Es el approval gate.

4.3 Las etapas del montaje (una a una)

A continuación se describe cada etapa, su misión en la fábrica y qué puede romperla.

- **Checkout:** El capataz (Jenkins) ordena traer los planos más recientes desde el almacén (Gitea). Si la dirección del almacén está mal escrita o faltan las llaves (credenciales), la cadena se detiene.
- **Unit tests:** Se enciende una máquina de control de calidad (un contenedor de Node.js) que verifica que las piezas pequeñas funcionan (los tests unitarios). Si alguna prueba falla o el cableado de dependencias (npm install) da problemas, el pipeline se rompe.
- **Build image:** Se forja la caja que contendrá la calculadora (la imagen Docker). Si la receta (Dockerfile) tiene un error de sintaxis o falta algún archivo, la forja no produce nada y el proceso se detiene.
- **Deploy staging:** La nueva caja se envía al taller de pruebas (staging). Primero se retira la caja vieja, luego se coloca la nueva. Si la cinta transportadora (la red Docker) no está operativa o el puerto de montaje está ocupado, el despliegue falla.
- **Smoke test staging:** Un robot inspector lanza un curl al punto de control de salud (/health) y espera la palabra mágica "healthy". Espera 10 segundos a que la calculadora arranque. Si no recibe respuesta, si el motor responde con un error (500) o si la palabra no aparece, el inspector grita "¡esto no sirve!" y la cadena se para.
- **Aprobar despliegue a producción:** Jenkins se congela. Muestra un cartel en su panel de control: "¿Staging funciona correctamente?" y dos botones: "Desplegar" y "Abortar". Mientras el capataz humano no pulse, la fábrica espera. Esta etapa nunca falla por sí misma; solo espera.
- **Deploy production:** Una vez recibida la orden, se retira la caja de la sala de ventas (producción) y se coloca la nueva, idéntica a la que ya funcionó en pruebas. Si falla, es por las mismas razones que en staging: problemas de red, puerto ocupado, etc.

4.4 ¿Cómo se entera Jenkins de que hay trabajo nuevo?

En nuestra fábrica, Jenkins no tiene un telefonillo que le avise al instante. En lugar de eso, envía un mensajero cada dos minutos a preguntar al almacén (Gitea): "¿Hay planos nuevos?". Este mecanismo se llama polling SCM y está configurado con la expresión `H/2 * * * *` (cada dos minutos, con un desfase aleatorio para no saturar el servidor). La desventaja es que, desde que un ingeniero hace git push, pueden pasar hasta dos minutos hasta que el pipeline arranque.

Nota: El método más eficiente son los webhooks: Gitea avisa a Jenkins en el mismo momento del git push. No lo usamos en el laboratorio para simplificar la configuración de red.

4.5 Limitaciones y simplificaciones de nuestro pipeline (Laboratorio)

- Usamos la etiqueta latest para la imagen Docker. En un entorno real, cada build generaría una imagen única (calculator-app:42).
- No implementamos rollback automático. Si el despliegue a producción falla, no hay un mecanismo que recupere automáticamente la versión anterior. Confiamos en que el approval gate evitará que versiones defectuosas lleguen a producción.
- El smoke test solo comprueba el endpoint /health. En un proyecto real se añadirían más pruebas significativas (integración, rendimiento, etc.).

5. DESPLIEGUE A PRODUCCIÓN Y ORQUESTACIÓN DE LA INFRAESTRUCTURA BASE

En nuestra fábrica, el taller de pruebas (staging) y la sala de ventas (producción) están separados. La nueva versión de la calculadora se prueba primero en el taller. Solo si pasa todas las comprobaciones y un capataz humano da su visto bueno, se promociona a la sala de ventas. Pero, ¿quién construye y mantiene el resto de la fábrica? ¿Quién se asegura de que el almacén (Gitea), el cerebro (Jenkins) y los paneles de control (Prometheus, Grafana, cAdvisor) estén siempre en pie y correctamente configurados? Esa es la misión de Terraform.

5.1 El approval gate: la barrera humana

Tras superar el smoke test en staging, el pipeline se detiene mediante la directiva input. Jenkins muestra un botón en su interfaz web y espera. El capataz humano revisa el panel de control (Grafana), comprueba que el taller funciona con normalidad y, solo entonces, pulsa "Desplegar". Si pulsa "Abortar", el build se cancela y la sala de ventas no se toca. Mientras espera, el ejecutor de Jenkins no consume recursos de CPU; simplemente está en pausa.

Esta barrera es el corazón del Continuous Delivery: entregamos la nueva versión hasta staging de forma automática, pero su salida a producción es un evento consciente y controlado.

5.2 Despliegue azul/verde simulado

En una fábrica real, cuando se quiere cambiar el producto en la sala de ventas sin cerrar la tienda, se usa un balanceador de carga: se conecta la nueva versión por un lado, se desconecta la vieja por el otro, y los clientes ni se enteran. Nuestro laboratorio no tiene balanceador, pero el aislamiento entre staging y producción emula la misma idea:

- Staging (el taller) recibe la nueva versión para validación.

- Producción (la sala de ventas) sigue sirviendo la versión anterior.
- Cuando se aprueba el cambio, el pipeline detiene y elimina el contenedor de producción, y lanza uno nuevo con la imagen aprobada.

La limitación es evidente: hay un microcorte de servicio durante los segundos que dura el reemplazo. En una fábrica real con balanceador, ese corte no existe. Pero la lógica de protección —nada va a producción sin pasar antes por pruebas— es exactamente la misma.

5.3 Rollback automático (una mejora opcional)

El pipeline básico no incluye rollback automático. Si el despliegue a producción falla, el responsable humano debe decidir qué hacer. Sin embargo, se puede ampliar el Jenkinsfile con un bloque que, tras desplegar en producción, vuelva a comprobar el endpoint /health. Si falla, que destruya el contenedor defectuoso y levante la versión anterior (la que estaba etiquetada como latest antes del despliegue). El build terminaría en error, pero los clientes no verían el fallo.

Esta mejora no está implementada en el laboratorio base para mantener la simplicidad, pero se menciona como un buen patrón para entornos reales.

5.4 Terraform: el orquestador de la infraestructura de soporte

Terraform no construye la calculadora, sino que levanta y mantiene el esqueleto de la fábrica: los servicios sin los cuales el pipeline no podría funcionar. Estos servicios son:

- Jenkins (el capataz)
- Gitea (el almacén de planos)
- Prometheus (los sensores)
- Grafana (el videowall)
- cAdvisor (el microscopio del kernel)

Terraform lee archivos de configuración escritos en HCL y los traduce en órdenes al demonio Docker. Su poder se basa en dos pilares:

- **Idempotencia:** ejecutar terraform apply diez veces produce el mismo resultado que hacerlo una vez. Si la infraestructura ya está como se pide, no hace nada. Si alguien toquetea un contenedor a mano, Terraform lo detecta y lo restaura.
- **Archivo de estado (terraform.tfstate)** : es la memoria de Terraform. Guarda en un JSON qué recursos creó y cómo están. Si se pierde o corrompe, Terraform pierde la noción de lo que existe.

Red y direcciones IP: Terraform crea una red Docker llamada LabCEFIRE y asigna a cada servicio de soporte una IP fija dentro de esa red. Así, Jenkins siempre sabe dónde está Gitea, y Prometheus sabe dónde están Jenkins y cAdvisor.

Persistencia: Terraform define dos volúmenes Docker que sobreviven a los reinicios de contenedores:

- **jenkins-data:** guarda todo el estado de Jenkins (jobs, credenciales, plugins).
- **gitea-data:** guarda los repositorios Git y la base de datos SQLite de Gitea.

Inyección de archivos (bind mounts): Terraform copia archivos de configuración directamente dentro de los contenedores cuando los crea. Por ejemplo:

- El **pipeline-config.xml** se monta en Jenkins para definir el job.
- El **prometheus.yml** se monta en Prometheus para configurar los objetivos de scraping.
- Los archivos de **provisioning** de Grafana (datasource, dashboard provider y el JSON del dashboard) se montan para que Grafana arranque ya configurado.

Gracias a esto, la fábrica arranca completamente operativa, sin necesidad de entrar a configurar nada a mano desde las interfaces web.

5.5 Separación de responsabilidades: Terraform vs Jenkins

Terraform NO gestiona los contenedores de la aplicación (**ud06-staging y ud06-production**). ¿Por qué? Porque esos contenedores cambian constantemente (cada build). Si Terraform los gestionara, se produciría un conflicto: Jenkins los pararía, borraría y recrearía, y Terraform, al detectar la diferencia, intentaría restaurarlos a su estado original.

La regla es clara:

- Terraform se encarga de la infraestructura estable (servicios de soporte).
- Jenkins se encarga del despliegue dinámico de la aplicación (staging y producción).

Esta separación es un ejemplo de orquestación híbrida, muy común en entornos reales.

6. OBSERVABILIDAD Y DIAGNÓSTICO DE FALLOS

Una fábrica sin medidores de temperatura, sin velocímetros y sin pantallas es una fábrica a ciegas. Si un build falla, necesitamos saber por qué. Si el taller de pruebas (staging) consume más RAM de la cuenta, necesitamos verlo antes de que la sala de ventas (producción) siga el mismo camino. Nuestro ecosistema de telemetría es el sistema nervioso que recorre cada rincón del laboratorio.

6.1 Prometheus: los sensores que todo lo vigilan

Prometheus no espera a que le envíen datos. Es un recolector activo: cada 15 segundos visita los puntos de control de sus objetivos y les arranca las métricas mediante HTTP. Este modelo se llama pull, y es opuesto al modelo push de otras herramientas. La ventaja es que Prometheus controla el ritmo y detecta por sí mismo si un objetivo ha dejado de responder.

En nuestra fábrica, Prometheus raspa la información de cuatro fuentes:

- **Jenkins:** gracias al plugin Prometheus, expone métricas como la duración de los builds, el resultado (éxito o fracaso) y la cola de trabajos.
- **cAdvisor:** expone métricas detalladas de cada contenedor: CPU, memoria, red y disco.
- **Staging y Producción:** la calculadora expone su propia métrica *app_version*, que permite saber qué versión está corriendo en cada entorno.

La configuración de Prometheus (qué objetivos rascar y cada cuánto) se define en un archivo **prometheus.yml** que Terraform inyecta al crear el contenedor. Así, los sensores llegan ya calibrados a la fábrica.

6.2 cAdvisor: el microscopio montado sobre el kernel

cAdvisor (Container Advisor) es un contenedor privilegiado que se conecta directamente al demonio Docker y lee del kernel de Linux las métricas reales de cada contenedor. Es como un microscopio que mira dentro de las máquinas sin desmontarlas.

Una sutileza importante: la métrica relevante para la memoria no es *container_memory_usage_bytes* (que incluye cachés inactivas del sistema de archivos) sino *container_memory_working_set_bytes*. Esta última no cuenta esas cachés y muestra lo que el contenedor realmente está usando. Usar la métrica equivocada es como diagnosticar una fiebre cuando el paciente solo tiene calor por estar al sol.

cAdvisor expone cientos de métricas con el prefijo *container_**, pero las que más nos importan son:

- *container_cpu_usage_seconds_total* (segundos de CPU consumidos)
- *container_memory_working_set_bytes* (memoria real)
- *container_network_receive_bytes_total* (tráfico de red)

6.3 Grafana: el videowall del capataz

Grafana es el videowall de la fábrica. No hay que configurarlo a mano: Terraform inyecta un archivo JSON con el dashboard ya definido. Cuando Grafana arranca, ya sabe qué mostrar.

El dashboard traduce los vectores de PromQL (el lenguaje de consulta de Prometheus) en paneles visuales:

- Estado del último build: un semáforo verde si fue éxito, rojo si falló.
- Duración del pipeline: una línea temporal que muestra si los builds se están alargando con el tiempo.
- Consumo de recursos: gráficas de CPU y RAM de Jenkins, Gitea, staging y producción, cortesía de cAdvisor.
- Versión desplegada: un indicador que dice "v1" o "v2" en cada entorno, gracias a la métrica *app_version*.

De un vistazo, el capataz humano sabe si la cadena está fluyendo o si hay un problema.

7. MÉTRICAS REALES DEL PANEL DE CONTROL

El videowall de nuestra fábrica (Grafana) no muestra conceptos abstractos, sino métricas concretas que los sensores (Prometheus) capturan cada pocos segundos. A continuación se describen las que hemos configurado en el laboratorio, agrupadas por su procedencia.

7.1 Versión desplegada en cada entorno (métrica de la aplicación)

La propia calculadora expone una métrica personalizada en su endpoint */metrics*:

app_version{version="1.0.0"} Prometheus la recoge de los contenedores de staging y producción. En Grafana tenemos dos paneles (uno para staging, otro para producción) que muestran la versión actual. Gracias a esto, se sabe de un vistazo si el taller de pruebas está ejecutando la v1 (suma) o la v2 (multiplicación).

7.2 Estado de la plataforma Git (Gitea)

Gitea expone métricas propias gracias a su configuración (GITEA__metrics__ENABLED=true). En el dashboard vemos dos indicadores:

- **Repositorios:** $sum(gitea_repositories)$. Nos dice cuántos proyectos hay almacenados.
- **Usuarios:** $sum(gitea_users)$. Nos dice cuántos desarrolladores tienen acceso.

Aunque en el laboratorio solo hay un repositorio (calculator-app) y un usuario (devops-admin), estas métricas preparan al alumno para entornos más grandes.

7.3 Salud del pipeline Jenkins

Jenkins, gracias al plugin Prometheus, expone dos contadores clave:

- **Total de builds:** $default_jenkins_builds_success_build_count_total + default_jenkins_builds_failed_build_count_total$. Es la suma de todos los pipelines ejecutados desde que Jenkins arrancó.
- **Builds fallidos:** $default_jenkins_builds_failed_build_count_total$. Un contador que solo aumenta cuando un build termina en rojo.

En el dashboard vemos ambos números. Si el total de builds es, por ejemplo, 42 y los fallidos son 3, sabemos que la tasa de fallos es de aproximadamente el 7%. El capataz puede fijar su propio umbral de alerta mental.

7.4 Consumo de recursos (CPU y RAM) por contenedor

cAdvisor es la fuente de estas métricas. En el dashboard hemos configurado dos gráficas de series temporales:

- **CPU (%):** $sum(rate(container_cpu_usage_seconds_total\{name=~"ud06.*"\}[1m])) \text{ by } (name) * 100$. Cada contenedor (Jenkins, Gitea, staging, producción, etc.) muestra su línea de color. Podemos ver si el cerebro de la fábrica (Jenkins) está saturado al 80% de CPU mientras construye una imagen, o si el taller de pruebas (staging) se dispara por un bucle infinito.
- **RAM (bytes):** $container_memory_usage_bytes\{name=~"ud06.*"\}$. Aquí mostramos la memoria que cada contenedor está consumiendo. Es importante recordar que esta métrica incluye cachés inactivas; si se quisiera la memoria realmente activa, se usaría $container_memory_working_set_bytes$ (pero en el laboratorio hemos dejado la más sencilla para no complicar).

Estas gráficas permiten detectar fugas de memoria o contenedores que crecen sin control.

7.5 ¿Qué métricas NO están en el panel y por qué?

El panel del laboratorio no incluye directamente:

- Duración del pipeline (aunque Jenkins la expone, no la hemos añadido al dashboard para no saturarlo).
- MTTR (tiempo de recuperación), porque no hay rollback automático y dependería de medidas manuales.
- Disponibilidad del servicio (sondas HTTP), porque el laboratorio se centra en el pipeline,

no en el SLA.

El alumno puede añadir estas métricas como ejercicio de ampliación, pero el panel base muestra lo esencial para operar la fábrica.

8. ★ BUENAS PRÁCTICAS EN LA IMPLEMENTACIÓN DE PIPELINES CI/CD

Después de recorrer toda la cadena de montaje, estas son las máximas que separan una fábrica funcional de una robusta. Algunas las hemos aplicado en el laboratorio; otras son mejoras que se pueden añadir.

8.1 Inmutabilidad absoluta

Nunca, bajo ninguna circunstancia, se entra por SSH a la sala de ventas (producción) a "arreglar" un archivo. El código se corrige en el almacén de planos (Git), se genera una nueva versión y el pipeline recrea la caja (contenedor) desde cero. La imagen que pasa las pruebas es la misma que llega a producción. Si no se cumple esto, no hay pipeline que valga.

8.2 Pipeline como código

El **Jenkinsfile** viaja con la aplicación, se versiona en Git y se revisa como cualquier otro código. No existe ninguna configuración manual en la interfaz web de Jenkins que no pueda reconstruirse desde el repositorio. Si el cerebro (Jenkins) muere y hay que recrearlo, Terraform levanta un nuevo contenedor con el mismo volumen, y el pipeline reaparece tal cual.

8.3 Idempotencia y determinismo

Ejecutar el mismo pipeline sobre el mismo **commit** debe producir el mismo resultado. Las imágenes base usan etiquetas fijas (por ejemplo, node:20-alpine, nunca latest). Las dependencias están bloqueadas con package-lock.json. Si hay aleatoriedad (como el H en el polling SCM), está controlada y es reproducible.

8.4 Separación de entornos

El taller de pruebas (staging) y la sala de ventas (producción) son gemelos en configuración, pero con variables de entorno distintas y (en un escenario real) redes separadas. Los datos en staging son sintéticos o anonimizados. Nunca se prueba con datos reales de usuarios.

8.5 Seguridad con mínimo privilegio

Las credenciales (como el usuario de Gitea) no viajan en el código fuente. Se almacenan cifradas en el almacén de Jenkins (credenciales) y se inyectan solo cuando son necesarias. El montaje del socket Docker en Jenkins (DooD) es una puerta blindada que solo debe abrirse en entornos de laboratorio controlados. En producción se usarían agentes remotos o DinD con recursos acotados.

8.6 Artefactos inmutables y etiquetado

En el laboratorio hemos usado la etiqueta latest por simplicidad, pero esto es una mala práctica en entornos reales. Cada build debe generar una imagen etiquetada con un identificador único (número de build o hash del commit), por ejemplo calculator-app:42. Así se puede hacer

rollback con certeza y se evitan confusiones. Además, esas imágenes deberían subirse a un registro privado (Harbor, ECR, etc.) y el pipeline desplegar siempre desde el registro, no reconstruyendo localmente.

8.7 Limpieza periódica de la fábrica

El uso intensivo de Docker genera imágenes huérfanas (dangling), volúmenes no utilizados y contenedores detenidos. Estos restos ocupan espacio en disco y pueden agotar los inodos del sistema de archivos. El laboratorio incluye scripts de destrucción (98-destroy-infra.sh) y se recomienda ejecutar periódicamente `docker system prune -f` para mantener la fábrica limpia.

8.8 Monitorización proactiva

El videowall de Grafana no solo es bonito, sino que permite establecer alertas. Por ejemplo, si la tasa de builds fallidos supera el 10%, Prometheus puede enviar una notificación. En el laboratorio no hemos configurado alertas, pero es el siguiente paso natural.

9. WEBGRAFÍA

- **Jenkins Pipeline Official Documentation**  <https://www.jenkins.io/doc/book/pipeline/>
 - La biblia del arquitecto de CI/CD. Todo lo que necesitas saber sobre la sintaxis declarativa, el uso de Groovy en el Sandbox, *stages*, *steps* y cómo gestionar el ciclo de vida de tu factoría digital.
- **Docker-out-of-Docker (DooD) Explained**  <https://devops.com/the-docker-out-of-docker-pattern/>
 - Artículo fundamental para entender por qué compartir el socket `/var/run/docker.sock` del Host con el contenedor de Jenkins es la forma más eficiente (y delicada a nivel de seguridad) de forjar nuevas imágenes sin los problemas de rendimiento de DinD (Docker-in-Docker).
- **Gitea - Git with a cup of tea**  <https://docs.gitea.com/>
 - Documentación oficial del servidor Git ligero que utilizamos como "Almacén de Planos". Indispensable para entender la configuración de Webhooks y la gestión de repositorios mediante API.